

Algoritmo y Complejidad

Trabajo - Especificación de Tipos Abstractos de Datos

Alumna: Magdalena Sanabria

1. TAD fracción

Dominio

Pares ordenados (numerador, denominador) de números enteros, con denominador distinto de cero

Operaciones

- crear(n: Entero, d: Entero) -> Fracción

Signatura: crear(n: Entero, d: Entero) -> Fracción

Precondición: $d \neq 0$.

Postcondición: el resultado representa la fracción n/d .

- numerador(f: Fracción) -> Entero

Signatura: numerador(f: Fracción) -> Entero

Precondición: f es una Fracción válida.

Postcondición: devuelve el numerador de f, sin modificar f.

- denominador(f: Fracción) -> Entero

Signatura: denominador(f: Fracción) -> Entero

Precondición: f es una Fracción válida.

Postcondición: devuelve el denominador de f, sin modificar f.

- sumar(f1: Fracción, f2: Fracción) -> Fracción

Signatura: sumar(f1: Fracción, f2: Fracción) -> Fracción

Precondición: f1 y f2 son Fracciones válidas.

Postcondición: el resultado representa la suma matemática de f1 y f2, expresada como una nueva Fracción.

- simplificar(f: Fracción) -> Fracción

Signatura: simplificar(f: Fracción) -> Fracción

Precondición: f es una Fracción válida.

Postcondición: el resultado es equivalente a f, con numerador y denominador coprimos.

- restar(f1: Fracción, f2: Fracción) -> Fracción

Signatura: restar(f1: Fracción, f2: Fracción) -> Fracción

Precondición: f1 y f2 son Fracciones válidas.

Postcondición: el resultado representa la resta matemática de f1 y f2, expresada como una nueva Fracción.

- multiplicar(f1: Fracción, f2: Fracción) -> Fracción

Signatura: multiplicar(f1: Fracción, f2: Fracción) -> Fracción

Precondición: f1 y f2 son Fracciones válidas.

Postcondición: el resultado representa la multiplicación matemática de f1 y f2, expresada como una nueva Fracción.

- sonIguales(f1: Fracción, f2: Fracción) -> Booleano

Signatura: sonIguales(f1: Fracción, f2: Fracción) -> Booleano

Precondición: f1 y f2 son Fracciones válidas.

Postcondición: devuelve verdadero si f1 y f2 representan el mismo valor racional, aunque estén expresadas con diferentes numeradores y denominadores; en caso contrario, devuelve falso.

2. TAD Punto2D

Dominio

Pares ordenados (x, y) de números reales que representan puntos en el plano cartesiano.

Operaciones

- crear(x: Real, y: Real) -> Punto2D

Signatura: crear(x: Real, y: Real) -> Punto2D

Precondición: x e y son números reales válidos.

Postcondición: el resultado representa el punto (x, y).

- coordenadaX(p: Punto2D) -> Real

Signatura: coordenadaX(p: Punto2D) -> Real

Precondición: p es un Punto2D válido.

Postcondición: devuelve la coordenada X de p, sin modificar p.

- coordenadaY(p: Punto2D) -> Real

Signatura: coordenadaY(p: Punto2D) -> Real

Precondición: p es un Punto2D válido.

Postcondición: devuelve la coordenada Y de p, sin modificar p.

- distancia(p1: Punto2D, p2: Punto2D) -> Real

Signatura: distancia(p1: Punto2D, p2: Punto2D) -> Real

Precondición: p1 y p2 son Puntos2D válidos.

Postcondición: devuelve la distancia euclidiana entre p_1 y p_2 .

- trasladar(p : Punto2D, dx : Real, dy : Real) $\rightarrow$ Punto2D

Signatura: trasladar(p : Punto2D, dx : Real, dy : Real) $\rightarrow$ Punto2D

Precondición: p es un Punto2D válido y dx y dy son números reales válidos.

Postcondición: el resultado representa un nuevo punto con coordenadas $(x + dx, y + dy)$, sin modificar el punto original p .